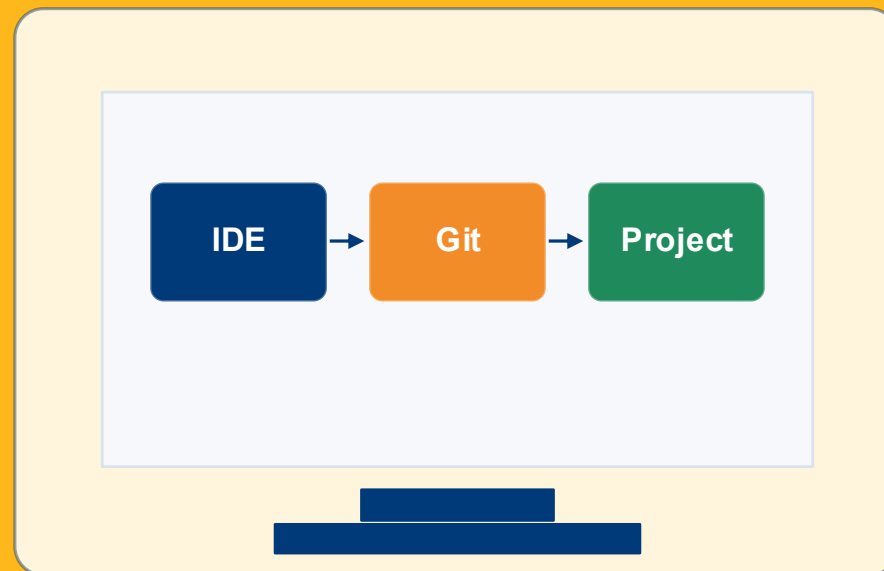


Tools for software development

Lab 1 - Development tools, Git and project setup



Lab output: working repository + project charter



Today's mission

By the end of the lab, every team should leave with a working repository and a clear starting point for the semester testing project.

1 Check and prepare tools

2 Practice Git workflow

3 Start semester project



Where Lab 1 fits in the course

The whole course is one continuous testing project. Lab 1 creates the foundation for all later testing activities.



Course alignment

Lab 1 supports practical skills in tools, collaboration, requirements understanding and project planning. These are needed for the tester role and for later manual and automated tests.

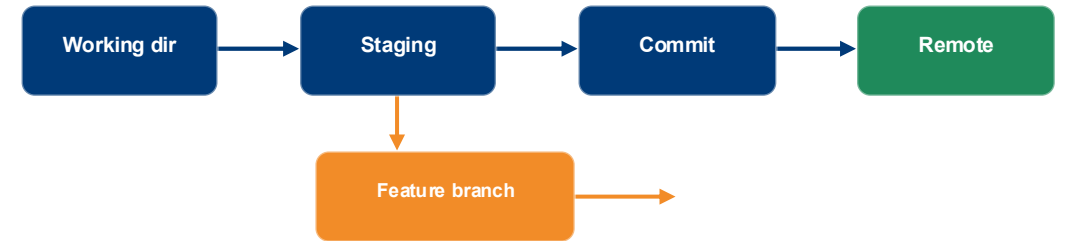
Assessment link

The repository, documentation and project artifacts created during labs form the continuous assignment part of the course. The final defense closes the project.



Learning outcomes for Lab 1

- Verify local development environment:
 - Git, Java and IDE.
- Use basic Git commands:
 - init, status, add, commit and log.
- Work with branches and understand why teams do not work directly on main.
- Resolve a simple merge conflict.
- Create the first version of team documentation in Markdown.
- Define tested application, initial scope and project risks.



Expected output:
team repository + README + project charter



Required tools

Before Git workflow, verify that the basic tools are available on each computer.

IDE

IntelliJ IDEA Community or Ultimate
Alternative: any IDE that supports
Java and Git

Git

Local Git installation
Access to GitHub, GitLab or school
Git

Java /Python

JDK installed / Python environment
Terminal can run java and javac

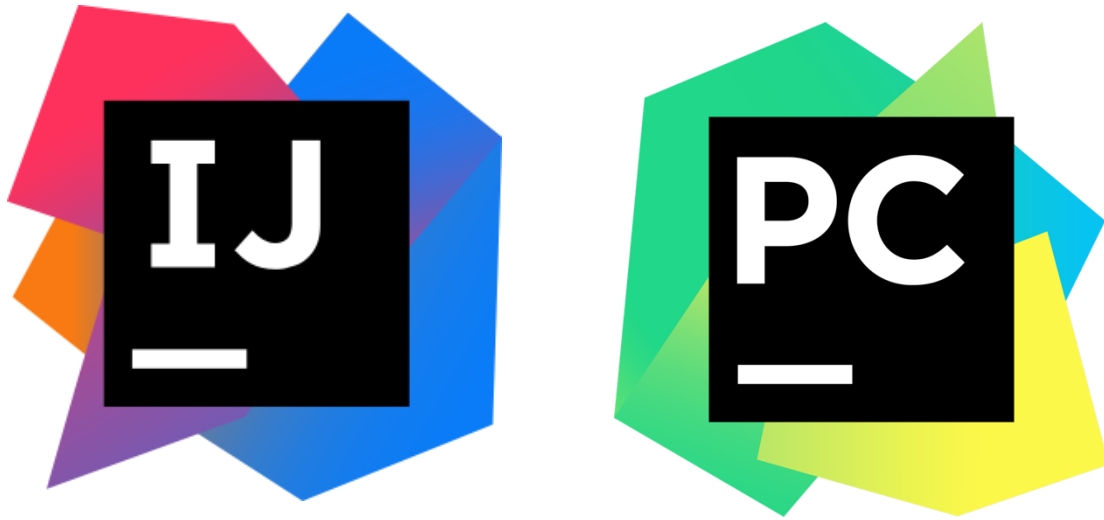
```
git --version  
java -version  
javac -version
```

**If any command fails, fix the environment
first. Later labs depend on this setup.**



Required tools

Choosing is on type of project, or team skills



IDE and project workspace

A clean workspace makes teamwork and testing easier. The repository should be readable before any tests exist.

Recommended workspace

- One folder for the course
- One repository per team
- Clear docs, tests and reports folders

IDE tasks today

- Open the repository
- Edit Markdown files
- Use integrated terminal
- Commit changes

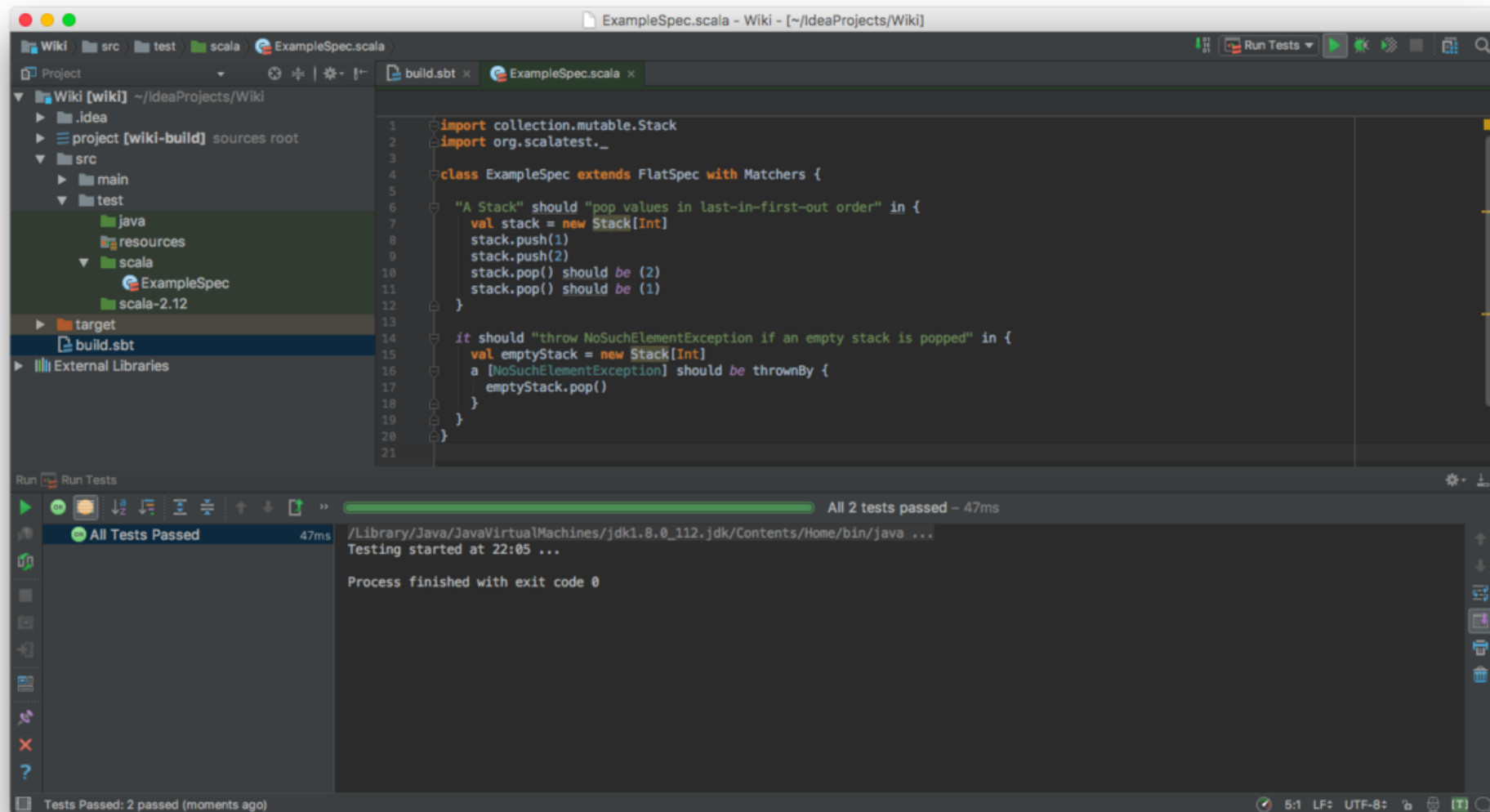
Quality habit

- Do not wait for the final report.
- Documentation starts on day one and grows with the project.

Instructor demo: create folder -> open in IDE -> create README.md -> commit.



IDE – Integrated Development Enviroment



Version

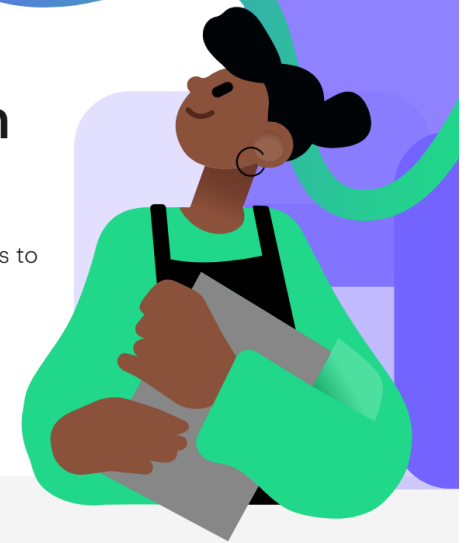
- freely available version of the Community
- for Java EE development - Ultimate
- Student packages:
- <https://www.jetbrains.com/student/>

 JETBRAINS Student Pack

Get career-ready with real developer tools

Access JetBrains IDEs, plugins, and hands-on courses to enhance your skills and build your job portfolio.

[Request now](#)



Options to get a free educational license



Official university email address



ISIC or ITIC card



GitHub Student Developer Pack [↗](#) account



Student's/teacher's ID or other official document



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

JetBrains Student Pack



ReSharper Ultimate

ReSharper, ReSharper C++, dotCover, dotTrace and dotMemory bundled in one license



IntelliJ IDEA Ultimate

A complete toolset for JVM-based web, mobile and enterprise development



AppCode

Smart iOS/macOS IDE



CLion

Cross-platform C/C++ IDE



DataGrip

Database and SQL IDE



GoLand

Capable and Ergonomic Go IDE



PhpStorm

IDE for Web & PHP



PyCharm

Powerful Python & Django IDE



Rider

Cross-platform .NET IDE



RubyMine

IDE for Ruby and Rails



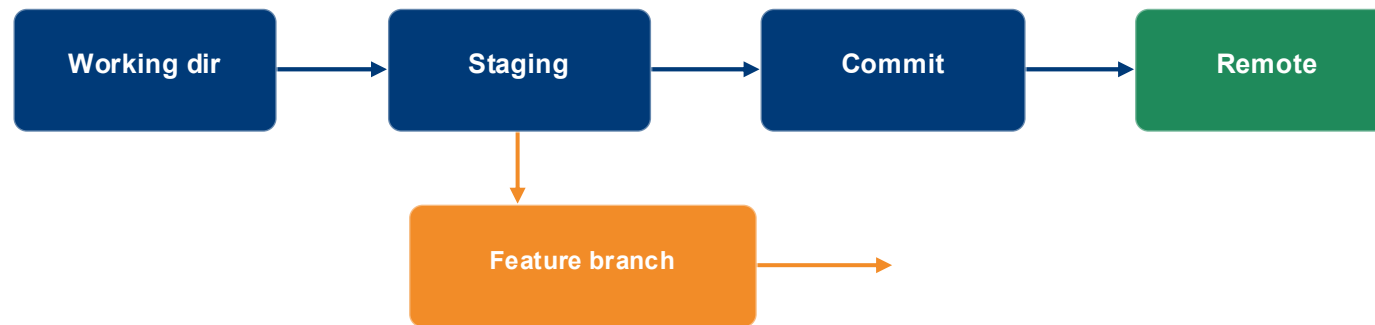
WebStorm

Smart JavaScript IDE



Version control concepts

Git records meaningful changes over time. A good repository tells the story of the project.



- Working directory - files you edit.
- Staging area - selected changes for the next commit.
- Commit - saved project snapshot with a message.
- Remote - shared repository used by the team.
- Branch - isolated line of work for a feature or change.



Version management - GIT



Version control

The meaning of software versioning, principles and benefits.

GIT

Basic functioning of the GIT versioning system.
Advantages and disadvantages.



GIT commands

Init, clone, fetch, commit, branch, merge, pull .

Examples

Examples of individual commands.



Basic Git warm-up

Individual task: create a local repository and make the first commit.

```
mkdir tsd-lab1-git-practice
cd tsd-lab1-git-practice
git init

# create README.md
git status
git add README.md
git commit -m "Add initial README"
git log --oneline
```

README.md

```
# TSD Lab 1 - Git Practice
```

```
Student: Your Name
```

```
This repository is used for practicing
basic Git workflow.
```

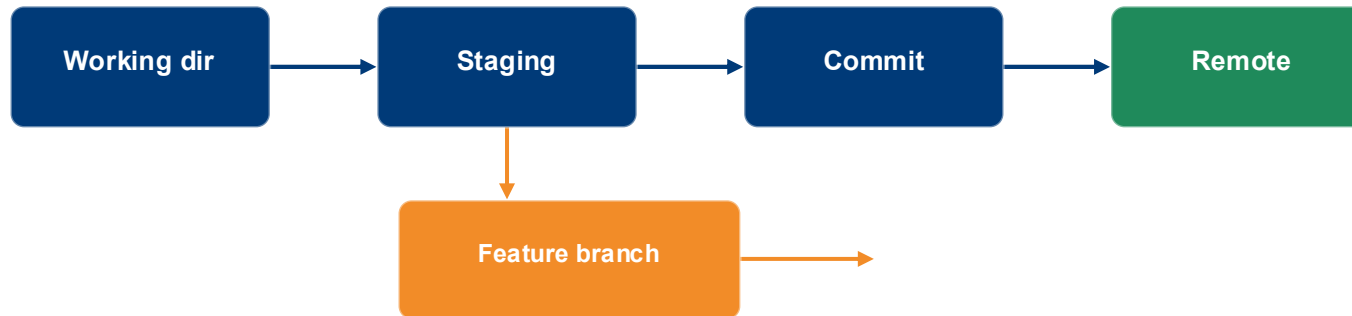
Check your result

git log --oneline should show one commit. The working tree should be clean after the commit.



Branching: do not work directly on main

Feature branches let several people work safely in parallel.



```
git switch -c feature/student-profile
# edit README.md
git add README.md
git commit -m "Add student profile section"

git switch main
git merge feature/student-profile
```

Branch rule for the project

Use one branch per task. Commit small meaningful changes. Merge only finished and reviewed work.



Merge conflict: expected, not catastrophic

A conflict happens when two branches change the same part of the same file.

```
<<<<<<< HEAD
# TSD Lab 1 - Git Practice - Main Version
=====
# TSD Lab 1 - Git Practice - Version A
>>>>>>> feature/version-a
```

```
Final title:
# TSD Lab 1 - Git Practice - Resolved Version

git add README.md
git commit -m "Resolve README title conflict"
```

Resolve in 4 steps

- 1 **Open the conflicted file**
- 2 **Choose or rewrite final text**
- 3 **Delete conflict markers**
- 4 **git add + git commit**



Exercise A: individual Git warm-up

Goal: create a local repository, commit README.md, create a feature branch and merge it back to main.

1 Create folder and run git init

2 Create README.md with student profile

3 Commit the initial version

4 Create feature branch

5 Add profile section and commit

6 Merge branch back to main

Timebox

15-20 minutes individual work. Ask for help after 3 minutes stuck.

Evidence

Show git log --oneline and clean git status.

Fast finishers

Repeat the workflow with a second branch and a second README section.



Exercise B: merge conflict practice

Goal: experience and resolve a controlled conflict in README.md.

```
git switch -c feature/version-a
# change first line in README.md
git add README.md
git commit -m "Change README title to version A"

git switch main
# change the same first line differently
git add README.md
git commit -m "Change README title in main"

git merge feature/version-a
```

Final expected title

```
# TSD Lab 1 - Git Practice - Resolved
Version
```

Finish with:

```
git add README.md
git commit -m "Resolve README title
conflict"
git log --oneline --graph --all
```



Semester project starts today

The team project will grow from documentation to manual and automated tests.



Team size

5-6 students per team. Assign clear responsibilities today.

Tested application

Prefer a stable web application with several user flows and, if possible, API endpoints.

Final artifacts

Documentation, manual test cases, automated tests, reports and presentation.

Today: choose the application, define scope and create the first project charter.



Recommended team roles

Roles are not titles only. Each role has a concrete responsibility and artifact.

Test Manager

coordination, deadlines, final scope

Test Designer

manual test cases and techniques

Automation Tester

Selenium and Robot tests

API Tester

Postman collection and API checks

Documentation Lead

README, report and presentation

Repository Lead

Git structure, branches, merges

In a 5-person team, one student can hold two smaller roles.



Team repository structure

Use the same structure in all teams. It makes assessment and later work easier.

```
tsd-2026-team-name/  
|-- README.md  
|-- docs/  
|   |-- team.md  
|   |-- project-charter.md  
|   `-- risks.md  
|-- manual-tests/  
|   `-- .gitkeep  
|-- automation/  
|   |-- selenium/  
|   |-- robot/  
|   `-- postman/  
`-- reports/  
    `-- .gitkeep
```

Why this structure?

docs stores project decisions, manual-tests stores test cases, automation stores tool-specific tests, reports stores results and screenshots.

.gitkeep

Git does not track empty folders. A small .gitkeep file allows teams to commit the planned folder structure.



README.md - first project page

README is the entry point for the project. It should answer what the project is and where to find artifacts.

```
# TSD 2026 - Semester Testing Project

## Team
Team name:

## Tested application
Application name:
Application URL:

## Project goal
The goal of this project is to design and execute
manual and automated tests for the selected web application.

## Tools planned
Git, JUnit, Selenium, Robot Framework, Postman
```

Good README

clear, concise, updated regularly, links to docs and reports

Bad README

empty, outdated, no team information, no scope, no instructions

Rule

A reviewer should understand the project in less than one minute.



Project charter - define the testing scope

The project charter is the first contract inside the team: what will be tested, why and what is out of scope.

1

Tested application

name, URL, short description

2

Business purpose

what the app is used for

3

Main users

visitor, user, admin, customer

4

Selected features

feature, description, priority

5

Out of scope

what will not be tested

6

Initial risks

risk, impact, mitigation

Today the charter may be version 0.1. It will be improved after test design and automation labs.



Example: Demo Web Shop

Use this as a model for your own application. Replace it with your real project.

Application

Demo Web Shop
A simple e-commerce website where users search products, view details and add items to cart.

Selected features

Product search - High
Product detail - Medium
Shopping cart - High
Login - High
Checkout form - Medium

Out of scope

Payment gateway
Real order processing
Admin dashboard
Performance testing

Initial risks

App changes during semester
Registration required
Dynamic UI elements
No public API available



Team Git workflow

Every team member must make at least one contribution through a branch.



```
git switch -c feature/member-name
# edit docs/team.md or docs/project-charter.md
git add docs/team.md
git commit -m "Add team member role"

git switch main
git merge feature/member-name
git push
```

Team rule

Before merging, quickly explain your change to another team member. This simulates a lightweight review.



Lab 1 checkpoint

Submit the repository link and make sure these files exist in main.

- README.md - project entry point
- docs/team.md - team members and roles
- docs/project-charter.md - tested app, scope and features
- docs/risks.md - first risks and mitigation ideas
- manual-tests/, automation/ and reports/ folders
- At least one branch created, committed and merged

Submission text

Team name:

Team members:

Repository URL:

Tested application:

Selected features:

Deadline

end of day 16 June 2026



Evaluation rubric for Lab 1

Lab 1 is a small checkpoint, but it sets up the whole semester project.

Team and roles defined

1 point

Functional Git repository with structure

1 point

README and basic documentation

1 point

Tested application and initial scope selected

1 point

Branch, commit and merge workflow demonstrated

1 point

Total: 5 points



Homework before Lab 2

Prepare your project so the next lab can focus on Java, JUnit and testing logic instead of basic setup.

- Improve the description of the tested application.
- List at least 5 features suitable for testing.
- Prepare first test data and example user accounts if needed.
- Clarify whether the application has API endpoints.
- Add open questions and risks to docs/risks.md.
- Make sure every team member can clone and open the repository.

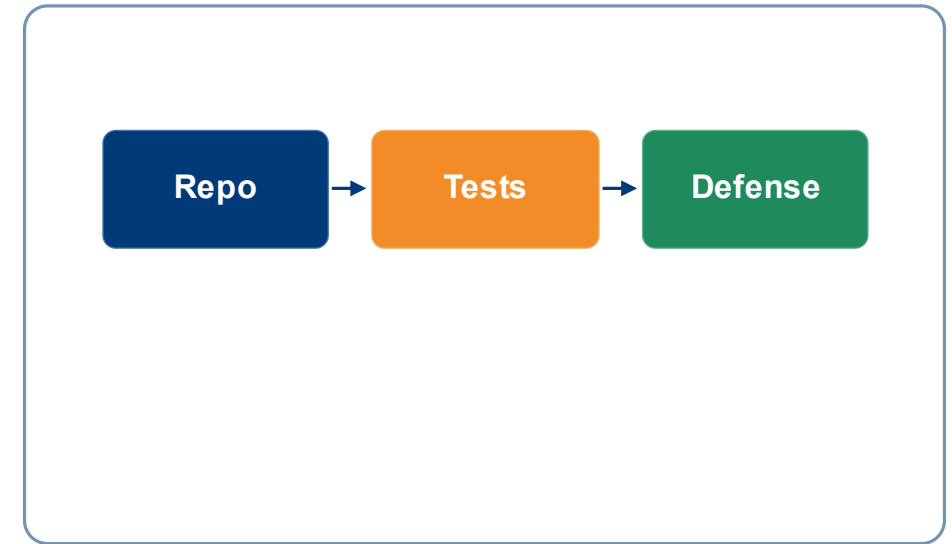
Next lab

Lab 2 - Basic types of tests, UnitTest and coverage

Bring to class

Repository, project charter and a concrete idea what logic can be unit-tested.





Thank you for your attention.

doc. Ing. **Jozef Kostolný**, PhD.
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
jozef.kostolny@fri.uniza.sk